

Deep Learning — Beginner's Visual Guide

ML

Contents

	ML Workflow	The 8 steps of any ML project
02	Evaluation Metrics	Accuracy, Precision, Recall, F1, AUC
03	Loss Curves	Diagnosing overfitting, learning rate problems
04	Optimizers	SGD, Momentum, Adam — how models learn
05	Regularisation	Stopping the model from memorising
06	Interpretability	Opening the black box — Grad-CAM, Integrated Gradients

■ Big picture analogy — training for a marathon

A deep learning project is like training for a marathon. You do not just lace up and run 42km on day 1. You follow a structured process: define your goal, measure progress, train gradually, adjust based on performance, and only race when truly ready.

The 8 steps of any ML project

1

Define the problem

What are you predicting? Cat vs dog? Cancer? House prices? This determines everything that follows.

2

Choose a measure of success

How will you know if it works? Accuracy? Recall? Picking the wrong metric is like measuring marathon success by sprint speed.

3

Prepare the data

Clean, normalise, encode categories, augment. Garbage in = garbage out.

4

Decide evaluation protocol

Split into train/validation/test. Never let the model see test data until the very end — like hiding the exam paper.

5

Monitor and diagnose training

Watch loss curves. Too fast? Too slow? Oscillating? Like checking your pulse mid-run.

6

Create a simple baseline

Try the simplest possible model first. If a coin flip beats you, something is very wrong.

7

Scale up — intentionally overfit

Make the model big enough to memorise training data perfectly. Proves it CAN learn. Then pull back.

8

Regularise and tune hyperparameters

Add dropout, weight decay, early stopping. Tune LR and batch size. Make it generalise.

Final model recipe (exam favourite):

Step 1: Use cross-validation on training data to find the best hyperparameters.

Step 2: Retrain on ALL data (train + validation) with those hyperparameters.

Step 3: Test ONCE on the held-out test set — that single number is your unbiased estimate.

Data preparation essentials

Normalisation

Scale values to [0,1] range.

$$x_{\text{norm}} = (x - \min) / (\max - \min)$$
Standardisation

Centre at 0, spread = 1 standard deviation.

$$x_{\text{std}} = (x - \text{mean}) / \text{std}$$
One-hot encoding

Convert categories to binary vectors. Avoids implying numerical order between categories.

Data augmentation

Create extra training examples by transforming existing ones: flips, rotations, noise, crops.

Must be domain-appropriate — do not flip a "6" digit into a "9".

Handling missing data

Option 1: Impute with mean or median.

Option 2: Use an out-of-range sentinel value.

Option 3: Discard the row/column.

■ The cancer detection scenario (exam favourite every year)

Imagine 1000 patients. 10 have cancer, 990 are healthy. A model that always says "healthy" achieves 99% accuracy — but it is completely useless and dangerous. This is exactly why we need better metrics than accuracy.

■ False Positive vs False Negative

False Positive = model says CANCER, patient is actually healthy → unnecessary fear and extra tests. False Negative = model says HEALTHY, patient actually has cancer → missed diagnosis. In medicine, False Negatives are far worse — so we prioritise HIGH RECALL (catch every real case).

Metric	Formula	When to use	Example scenario
Accuracy	Correct / Total	Only when classes are balanced in size	MNIST digit recognition — roughly equal samples per digit
Precision	$TP / (TP + FP)$	When false positives are costly	Spam filter — better to miss spam than delete real emails
Recall	$TP / (TP + FN)$	When false negatives are costly	Cancer detection — better safe than sorry
F1 Score	$2 \times P \times R / (P + R)$	Balanced trade-off, but report P and R separately too	Same F1 can hide terrible precision OR terrible recall
AUC-ROC	Area under ROC curve	Threshold-independent comparison of models	Comparing two classifiers on imbalanced medical data

Why cannot we use accuracy as a loss function?

Accuracy only changes when a prediction FLIPS. Tiny weight changes leave it completely flat. Gradient = 0 almost everywhere — gradient descent cannot work on a flat surface. We use cross-entropy loss instead (smooth and differentiable) and trust it tracks accuracy.

Loss function reference by task

Task	Output activation	Loss function	PyTorch
Binary classification	Sigmoid	Binary Cross-Entropy	<code>nn.BCELoss()</code> or <code>nn.BCEWithLogitsLoss()</code>
Multi-class	None (raw logit)	Cross-Entropy	<code>nn.CrossEntropyLoss()</code> (includes <code>LogSoftmax!</code>)
Regression	Linear / None	Mean Squared Error	<code>nn.MSELoss()</code>

Exam trap: `nn.CrossEntropyLoss` already applies `LogSoftmax` internally. NEVER add a `Softmax` layer before it in your `forward()` method — you would be applying softmax twice, which is mathematically wrong.

■ What is a loss curve?

A loss curve shows how your model's error changes over training epochs. Train loss = how well it does on data it has seen. Val loss = how well it does on brand new unseen data. The GAP between them is the most important thing to watch.

Symptom	Cause	What it looks like	Fix
Loss bounces wildly or explodes	Learning rate too HIGH	Val loss and train loss oscillate or shoot upward	Reduce LR by 10x
Loss decreases painfully slowly	Learning rate too LOW	Flat line barely moving downward	Increase LR or use Adam
Smooth steady decrease — both curves close	Learning rate just right	Both curves decline and stay near each other	This is what you want!

Underfitting vs Overfitting

Underfitting (too simple)	Overfitting (memorising)
Both train AND val loss are high throughout training.	Train loss is very low. Val loss is HIGH or rising.
The model has not learned enough.	The model has memorised training data.
Analogy:	Analogy:
Like studying with a 5-word summary — you do not know	Like memorising last year's exam answers — fails on any new question.
Fixes:	Fixes:
Use a bigger/deeper model. Train for more epochs. Add	More data. Dropout. Weight decay. Early stopping. Simpler model.

4 ways to fix overfitting

Fix	What it does	When to use first
Get more training data	More variety forces the model to learn real patterns, not memorise specific examples. Use data augmentation if collecting data is impossible.	Always — the most reliable fix
Add Dropout	Randomly disable neurons during training (e.g. 50%). Forces redundancy. Removed at test time.	After baseline is working

Weight Decay (L2)	Penalises large weights. Keeps model simple. Add <code>weight_decay=0.001</code> to your optimizer.	Almost always — very cheap
Early Stopping	Monitor val loss. Stop when it stops improving. Save the best checkpoint.	Always easy to add

■ The loss surface — a hilly landscape

Imagine you are blindfolded on a hilly landscape. Your goal is to reach the lowest valley (minimum loss). You can only feel the slope under your feet (the gradient). Different optimizers are different strategies for navigating this landscape.

Optimizer	Core idea	Update rule	Helps with
Plain SGD	Take a step in the direction of steepest descent. No memory.	$w = w - lr \times \text{gradient}$	Simple problems. Zigzags in narrow valleys.
SGD + Momentum	Build up velocity like a rolling ball. Remember direction from past steps.	$v = \text{beta} \times v_{\text{prev}} + \text{gradient}$ $w = w - lr \times v$	Narrow ravines. Plateaus. Noisy gradients. Typical beta = 0.9.
Adam	Momentum + each parameter gets its own adaptive learning rate based on history of its gradient.	Uses 1st and 2nd moment estimates with bias correction.	Default choice. Works without much tuning. Faster convergence generally.

Momentum — the rolling ball analogy

A ball rolling downhill builds up speed in a consistent direction. It does not react to every tiny bump — it smooths them out and keeps moving forward.

What momentum helps with:

1. Narrow valleys — dampens the zigzag across the walls, accelerates along the floor.
2. Plateaus — builds up speed through flat regions where gradient is tiny.
3. Noisy gradients — averages out random noise in mini-batch estimates.

$$v = 0.9 \times v_{\text{prev}} + \text{gradient}$$

$$w = w - lr \times v$$

Beta = 0.9 means: 90% old direction + 10% new gradient signal.

Adam — smart ball with GPS

Adam combines momentum with adaptive learning rates. Each weight gets its own learning rate based on how active it has been.

Two key innovations over plain SGD:

1. MOMENTUM: remembers the direction of past gradients, smooths noisy updates.
2. ADAPTIVE LR: weights with big gradients take small steps; weights with small gradients take bigger steps. Like a navigation system speeding up on motorways and slowing in tight streets.

Weight initialisation — why all-same is catastrophic:

If all weights = 0.1, every neuron computes the same thing, gets the same gradient, and updates identically forever. They can never specialise. Always use random initialisation.

■ The core problem — memorisation vs learning

Without regularisation, a powerful model will simply memorise the training data — like writing all answers on your palm before an exam. Perfect score in training, complete failure on anything new. Regularisation forces the model to find general patterns that work even on unseen data.

Explicit regularisation — adds a term to the loss

Method	Modified loss	Effect	Key property
L2 (Weight Decay)	$L = L_{\text{data}} + \lambda \times \sum(w^2)$	Keeps weights small and smooth.	Does NOT force weights to exactly zero. Just nudges them smaller. In PyTorch: <code>weight_decay</code> param in optimizer.
L1 (LASSO)	$L = L_{\text{data}} + \lambda \times \sum(w)$	Encourages sparsity.	Pushes many weights to EXACTLY zero. Good for feature selection — the model learns which inputs to completely ignore.

Implicit regularisation — no term added to loss

Technique	How it works	Simple analogy
Dropout	During each training step, randomly zero each neuron with probability p . At test time: no dropout, weights scaled by $(1-p)$.	Like studying while some of your notes are randomly hidden — forces you to learn multiple ways to arrive at the answer.
Early Stopping	Monitor val loss. When it stops improving, stop training and save the model from that best checkpoint.	Like knowing when to stop revising — more cramming past a point is actually harmful.
Data Augmentation	Generate extra training examples via label-preserving transformations: flips, rotations, noise, crops.	Like practising maths problems with slightly different numbers — builds real understanding, not memorisation.
Ensemble / Bagging	Train multiple different models on different data subsets. Average their predictions.	Like asking 10 doctors for a second opinion and going with the majority vote.

Bias-Variance Tradeoff — the fundamental tension

Expected Test Error = $\text{Bias}^2 + \text{Variance} + \text{Noise}$ (irreducible)

BIAS = systematic error from a model that is too simple. It misses real patterns. (Underfitting)

VARIANCE = how much the model changes across different training sets. Too sensitive to the specific data it saw. (Overfitting)

NOISE = inherent randomness in data — no model can fix this.

Making the model more complex reduces bias but increases variance. Adding regularisation reduces variance but may increase bias. The goal is the sweet spot between both.

■ The husky vs wolf problem

A model correctly classifies Huskies as wolves 80% of the time. Using Grad-CAM, researchers discovered it was looking at the SNOW IN THE BACKGROUND, not the animal. Without interpretability tools this dangerous bias is completely invisible.

Four families of interpretability methods

Family	Core idea	Examples
Weights / Activations	Look directly at what filters look like. Visualise activations as heatmaps.	First-layer filter visualisation, t-SNE of embeddings
Perturbation-based	Poke the input and see how the output changes. Cover regions and measure drops.	Occlusion analysis, LIME
Gradient-based	Use backpropagation to measure how sensitive each input is to the output.	Saliency Maps, SmoothGrad, Integrated Gradients, LRP
Class Activation Maps	Highlight which spatial regions of the image influenced the class decision most.	CAM, Grad-CAM, Guided Grad-CAM

The three gradient-based methods in detail

Vanilla Saliency

WHAT IT DOES: Compute the gradient of the output class score with respect to each input pixel. High gradient = changing that pixel changes the prediction a lot.

ANALOGY: Poking each pixel gently and measuring how much the model flinches.

ADVANTAGE: Very fast (single backprop pass). Simple to implement.

LIMITATION: Noisy output. Confuses sensitivity with importance. Fails at saturation (gradient = 0 even for important pixels).

SmoothGrad

WHAT IT DOES: Add random Gaussian noise to the input N times. Compute saliency for each noisy version. Average the results.

ANALOGY: Asking the same question 50 times with slightly different phrasing — consistent answers reveal the real signal.

ADVANTAGE: Visually cleaner, less noisy maps. Better stability.

LIMITATION: N times more expensive than vanilla saliency. Still a sensitivity measure, not true attribution.

Integrated Gradients

WHAT IT DOES: Start from a blank baseline (all-black image). Gradually morph to the real input. Integrate (accumulate) gradients along that entire path. Each pixel gets credit for the entire journey.

ANALOGY: To understand why a house is expensive, trace the value added step by step as you add each feature — not just look at the final price.

ADVANTAGE: Satisfies COMPLETENESS (attributions sum to the total output difference). Fixes the saturation problem. Principled and rigorous.

LIMITATION: Requires multiple forward passes (one per interpolation step). Choice of baseline affects results.

Sensitivity vs Attribution — the key distinction

SENSITIVITY (Vanilla Saliency): "If I wiggle this pixel by a tiny amount, does the output change?" — local, point-in-time measurement. A pixel can be critically important but show zero gradient if we are in a saturation zone.

ATTRIBUTION (Integrated Gradients): "How much did this pixel CONTRIBUTE to the output, compared to a neutral baseline?" — global, path-integrated measurement. More honest about what actually mattered for the prediction.

Method comparison at a glance

Method	Type	Speed	Resolution	Class-specific?
Vanilla Saliency	Gradient	Fast	Pixel-level	Yes
SmoothGrad	Gradient (averaged)	Medium	Pixel-level	Yes
Integrated Gradients	Gradient (path)	Medium	Pixel-level	Yes
LRP	Propagation	Medium	Pixel-level	Yes
Occlusion	Perturbation	Slow	Patch-level	Yes
LIME	Perturbation	Slow	Superpixel	Yes
Grad-CAM	Gradient + Activation	Fast	Feature map	Yes